

Deploying SonarQube and Scanning a Vulnerable Repository

Batch: DITISS

Module: Security Concepts

Task: Static Code Analysis with SonarQube on a Vulnerable Web Application

Due Date: 30/6/2026

This lab focuses on deploying SonarQube and scanning the Damn Vulnerable Python Web App (DVPWA) repository.

Tasks:

Part 1: Deploy SonarQube dashboard & Access the Dashboard

Part 2: Prepare the Vulnerable Repository

Part 3: Create a SonarQube Project with Token

Part 4: Set Up and Run the SonarScanner

Part 5: Run the Scan

Part 6: Review Scan Results

Submission:

Report: PDF with screenshots (dashboard, key issues)

Configurations

docker pull sonarqube

```
root ~  
❯ docker pull sonarqube  
Using default tag: latest  
latest: Pulling from library/sonarqube  
4f4fb700ef54: Pull complete  
ca2678b20700: Pull complete  
93eeac639cb8: Pull complete  
6f002909f8b6: Pull complete  
60360c32d845: Pull complete  
2a37e4a03506: Pull complete  
5a66f521929b: Pull complete  
1bd28c2fdfb3: Download complete  
fddd18ff8622: Download complete  
Digest: sha256:1ddbad7b16f754bc3985f336b8ef3ebe885c1655101bdb9cb4156c9560526f58  
Status: Downloaded newer image for sonarqube:latest  
docker.io/library/sonarqube:latest
```

```
root ~  
❯ docker images | grep sonar  


|                  |              |        |        |
|------------------|--------------|--------|--------|
| sonarqube:latest | 1ddbad7b16f7 | 2.53GB | 1.08GB |
|------------------|--------------|--------|--------|


```

docker network create sonar-network

```
root ~  
❯ docker network create sonar-network  
e32e372c1591950f4072a8ff0060b817e9bfdd64848df9e0985b313f41f82be7
```

```
docker run -d \  
--name sonarqube-db \  
--network sonar-network \  
-e POSTGRES_USER=sonar \  
-e POSTGRES_PASSWORD=sonar \  
-e POSTGRES_DB=sonarqube \  
postgres:alpine
```

```
root ~  
❯ docker run -d \  
--name sonarqube-db \  
--network sonar-network \  
-e POSTGRES_USER=sonar \  
-e POSTGRES_PASSWORD=sonar \  
-e POSTGRES_DB=sonarqube \  
postgres:alpine  
2ea83871f375434e5292814303f253966784d407151c2d89b6d24a42247f083c
```

This creates a PostgreSQL container with:

Variable	Value
Database	sonarqube
Username	sonar
Password	sonar

root ~				
 docker images grep post				
postgres:alpine	1b1689b20d16	433MB	121MB	U

```
docker run -d \
--name sonarqube \
--network sonar-network \
-p 9000:9000 \
-e SONAR_JDBC_URL=jdbc:postgresql://sonarqube-db:5432/sonarqube \
-e SONAR_JDBC_USERNAME=sonar \
-e SONAR_JDBC_PASSWORD=sonar \
sonarqube
```

root ~	
 docker run -d \	
--name sonarqube \	
--network sonar-network \	
-p 9000:9000 \	
-e SONAR_JDBC_URL=jdbc:postgresql://sonarqube-db:5432/sonarqube \	
-e SONAR_JDBC_USERNAME=sonar \	
-e SONAR_JDBC_PASSWORD=sonar \	
sonarqube	
239c033d42e645f4a492ad6f1da69a7d7e35f56ef17a470b4d3a472a52ade839	

docker ps

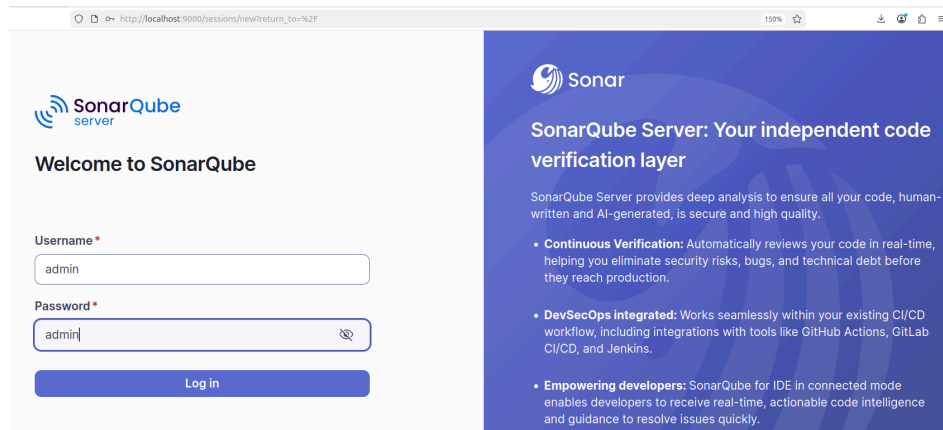
root ~							
 docker ps							
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS		NAMES
239c033d42e6	sonarqube	"/opt/sonarqube/dock..."	7 minutes ago	Up 7 minutes	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp		sonarqube
2ea83871f375	postgres:alpine	"docker-entrypoint.s..."	8 minutes ago	Up 8 minutes	5432/tcp		sonarqube-db

Ip -br a

root ~	
 ip -br a	
lo	UNKNOWN 127.0.0.1/8 ::1/128
ens33	UP 192.168.74.135/24 fe80::20c:29ff:feda:268/64
docker0	DOWN 172.17.0.1/16 fe80::ec3f:80ff:fef2:7e33/64
br-e32e372c1591	UP 172.18.0.1/16 fe80::585a:1eff:fed7:b2d/64
veth432d389@if2	UP fe80::acc7:a2ff:fe21:c450/64
vethf581871@if2	UP fe80::647a:1aff:fe08:8ec6/64

<http://localhost:9000/>

(default user:admin pswd:admin)



Username *

admin

Password *

admin

Log in

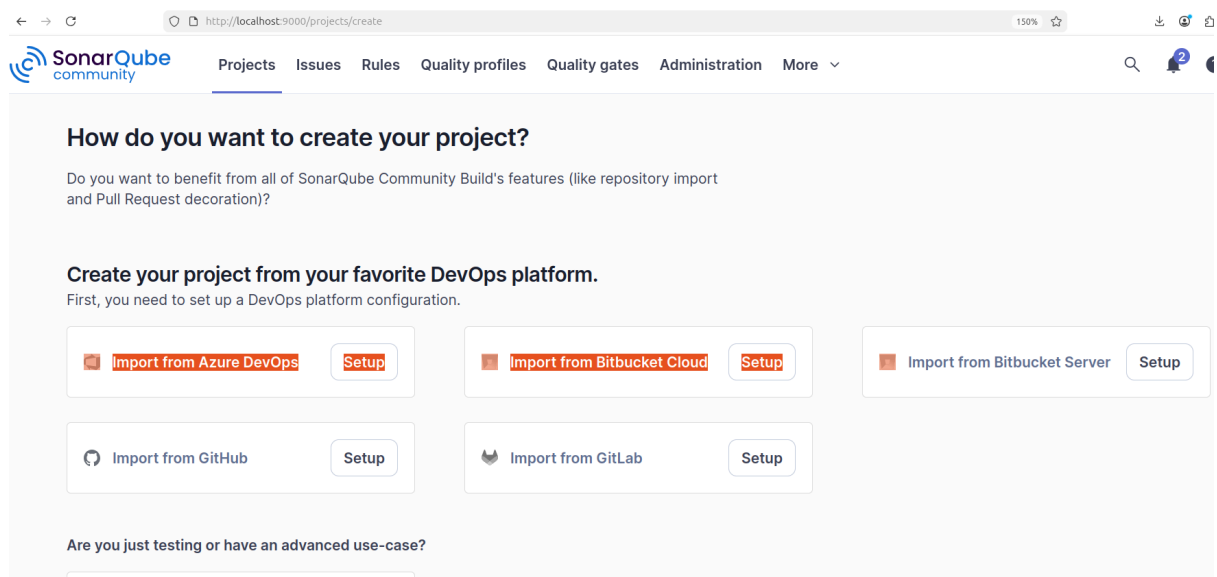
Sonar

SonarQube Server: Your independent code verification layer

SonarQube Server provides deep analysis to ensure all your code, human-written and AI-generated, is secure and high quality.

- **Continuous Verification:** Automatically reviews your code in real-time, helping you eliminate security risks, bugs, and technical debt before they reach production.
- **DevSecOps integrated:** Works seamlessly within your existing CI/CD workflow, including integrations with tools like GitHub Actions, GitLab CI/CD, and Jenkins.
- **Empowering developers:** SonarQube for IDE in connected mode enables developers to receive real-time, actionable code intelligence and guidance to resolve issues quickly.

Set new password (ex: Admin@123456789) then Project desktop will come



← → ↺

http://localhost:9000/projects/create

150%

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

How do you want to create your project?

Do you want to benefit from all of SonarQube Community Build's features (like repository import and Pull Request decoration)?

Create your project from your favorite DevOps platform.

First, you need to set up a DevOps platform configuration.

Import from Azure DevOps Setup

Import from Bitbucket Cloud Setup

Import from Bitbucket Server Setup

Import from GitHub Setup

Import from GitLab Setup

Are you just testing or have an advanced use-case?

Next: Part 2 Prepare the Vulnerable Repository

The assignment specifies using the **Damn Vulnerable Python Web Application (DVPWA)** as the target for analysis.

On the SonarQube server, clone the repository:

```
git clone https://github.com/anxolerd/dvpwa.git
```

Move into the project directory:

```
cd dvpwa
```

Verify the files:

```
ls -la
```

You should see files such as:

```
app.py
requirements.txt
templates/
static/
config.py
```

```
root ~
❯ git clone https://github.com/anxolerd/dvpwa.git
Cloning into 'dvpwa'...
remote: Enumerating objects: 123, done.
remote: Total 123 (delta 0), reused 0 (delta 0), pack-reused 123 (from 1)
Receiving objects: 100% (123/123), 992.95 KiB | 309.00 KiB/s, done.
Resolving deltas: 100% (41/41), done.
root ~
❯ cd dvpwa/
root ~/dvpwa
❯ ls
config          Dockerfile.app  LICENSE         README.rst     requirements.txt  sqli
docker-compose.yml Dockerfile.db   migrations      recreate.sh    run.py
```

Part 3: Create a SonarQube Project & Generate Authentication Token

Objective

Create a project in SonarQube that will receive the scan results and generate an authentication token for SonarScanner.

Step 1: Open SonarQube Dashboard

Open your browser: <http://localhost:9000>

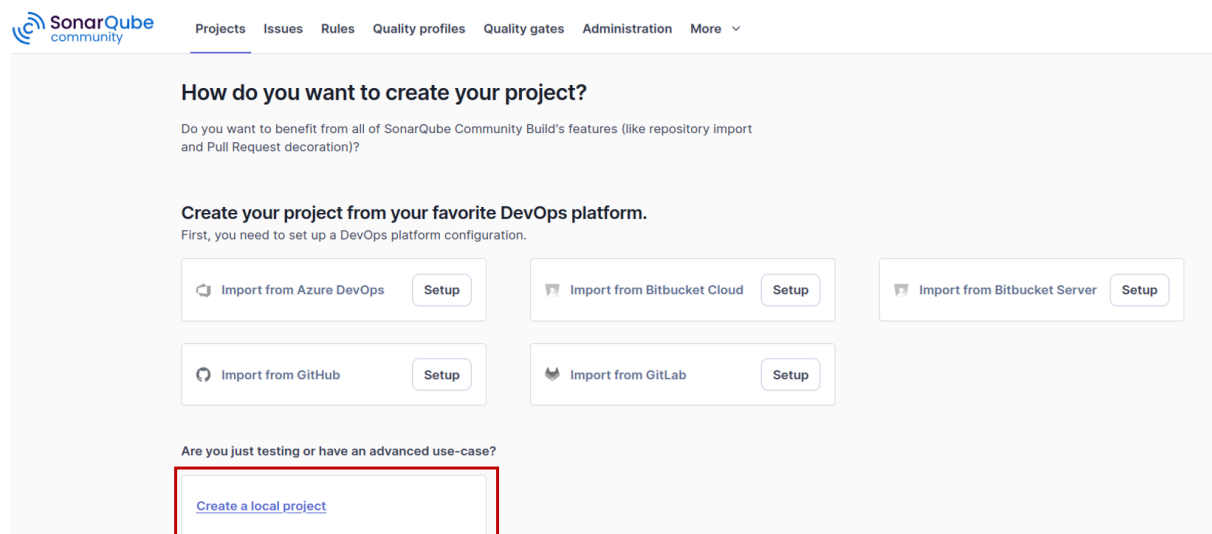
Login using your administrator account.

Step 2: Create a New Project

Click: **Create New Project**

or

Projects → Create Project



Step 3: Enter Project Details

Example:

Field	Value
Project Display Name	DVPWA
Project Key	dvpwa

Click **Set Up**.

1 of 2

Create a local project

Project display name * ⓘ
DVPWA

Project key * ⓘ
dvpwa

Main branch name *
main

The name of your project's default branch [Learn More](#) ⓘ

CancelNext

Step 4: Select Project Type

Custom

2 of 2

Set up new code for project

The new code definition sets the criteria for what's considered new code, allowing you to focus on the most recent changes in your project.

This setting allows project administrators to customize the new code definition for their project. [Learn more in documentation](#) 

Follows the instance's default

Current default: Previous version

Is custom

Choose whether it should be the **previous version**, a **number of days** or a **reference branch**

☐ Previous version

Any code that has changed since the previous version is considered new code. Recommended for **projects following regular versions or releases**.

☒ Number of days

Any code that has changed in the last x days is considered new code. If no action is taken on a new issue after x days, this issue will become part of the overall code. Recommended for **projects following continuous delivery**.

Define the number of days *

☐ Reference branch

Choose a reference branch as the baseline for the new code. Recommended for **projects using feature branches**.

[Back](#) [Create project](#)

Locally



ProjectsIssuesRulesQuality profilesQuality gatesAdministrationMore

DVPWA / Analysis Method

Analysis Method

Use this page to manage and set-up the way your analyses are performed.

How do you want to analyze your repository?

 [With Jenkins](#)

 [With GitHub Actions](#)

 [With GitLab CI](#)

 [With Azure Pipelines](#)

[Locally](#)

Use this for testing or advanced use-case.
Other modes are recommended to help you set up your CI environment.

Step 5: Generate Token

SonarQube will prompt you to create a token.

Example:

Token Name: scanner-token

Click

Generate

1 Provide a token

scanner-token: `sqp_961148e6ee44eede4e4e1b4a4d6af0ad23ff060a` 

The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point in time in your [user account](#) .

Continue

You'll receive a token similar to:

`sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXX`

Ex: `sqp_961148e6ee44eede4e4e1b4a4d6af0ad23ff060a`

 **Copy it immediately.** SonarQube only displays it once.

Step 6: Select Language

Choose

Python

and then select

Other (SonarScanner)

SonarQube will display example scanner commands. You can close that page, as you'll configure the scanner manually.

2 Run analysis on your project

What option best describes your project?

Maven

Gradle

JS/TS & Web

.NET

Python

Other (for Go, PHP, ...)

Install the Scanner for Python projects

```
pip install pysonar
```

Please visit the [official documentation of the Scanner for Python projects](#) for more details.

Execute the Scanner

Running a SonarQube analysis with the Scanner for Python projects is straightforward. You just need to run the following command in y

```
pysonar \  
--sonar-host-url=http://localhost:9000 \  
--sonar-token=sqp_961148e6ee44eede4e4e1b4a4d6af0ad23ff060a \  
--sonar-project-key=dvpwa
```

Please visit the [official documentation of the Scanner for Python projects](#) for more details.

pip install pysonar

pysonar \

--sonar-host-url=http://localhost:9000 \

--sonar-token=sqp_961148e6ee44eede4e4e1b4a4d6af0ad23ff060a \

--sonar-project-key=dvpwa

Use the SonarScanner Docker image

If you're already using Docker for SonarQube, you don't need to install the scanner manually.

`docker pull sonarsource/sonar-scanner-cli:latest`

```
root ~/dvpwa
❯ docker pull sonarsource/sonar-scanner-cli:latest
latest: Pulling from sonarsource/sonar-scanner-cli
95d605a7f04b: Pull complete
c6daf6f5f632: Pull complete
d2c57c9b2126: Pull complete
4f4fb700ef54: Pull complete
6d8e9a0f6859: Pull complete
c7bc63b14ec9: Pull complete
Digest: sha256:23ca0f137965d9dff2198074043fd48d386280bc5d0ccac8c8349cea4cf096a9
Status: Downloaded newer image for sonarsource/sonar-scanner-cli:latest
docker.io/sonarsource/sonar-scanner-cli:latest
```

Then run:

```
docker run --rm \
--network host \
-v $(pwd):/usr/src \
sonarsource/sonar-scanner-cli \
-Dsonar.projectKey=dvpwa \
-Dsonar.sources=. \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=YOUR_TOKEN
```

Replace **YOUR_TOKEN** with the token you generate from the SonarQube dashboard.

```
root ~/dvpwa
❯ docker run --rm \
--network host \
-v $(pwd):/usr/src \
sonarsource/sonar-scanner-cli \
-Dsonar.projectKey=dvpwa \
-Dsonar.sources=. \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=sqp_961148e6ee44eede4e4e1b4a4d6af0ad23ff060a
14:36:36.012 INFO Scanner configuration file: /opt/sonar-scanner/conf/sonar-scanner.properties
14:36:36.017 INFO Project root configuration file: NONE
14:36:36.040 INFO SonarScanner CLI 8.0.1.6346
14:36:36.048 INFO Linux 6.17.0-35-generic amd64
14:36:36.543 INFO Communication with SonarQube: Community Build 26.6.0-123539
```

Open your project

You should see a project named:

dvpwa

or directly open:

<http://localhost:9000/dashboard?id=dvpwa>

The screenshot shows the SonarQube dashboard for a project named 'dvpwa'. The dashboard is divided into two main sections: 'Overview' and 'Issues'.

Overview Section:

- Navigation: DVPWA / Overview
- Summary: 7.7k Lines of Code • Version not provided • Last analysis 2 minutes ago
- Alert: The last analysis has warnings. [See details](#)
- Metric Cards:
 - Security:** 3 Open issues (Grade B)
 - Reliability:** 17 Open issues (Grade D)
 - Maintainability:** 65 Open issues (Grade A)
 - Accepted issues:** 0 (Valid issues that were not fixed)
 - Coverage:** 0.0% (On 351 lines to cover)
 - Duplications:** 0.0% (On 20k lines)
 - Security Hotspots:** 3 (Grade E)

Issues Section:

- Navigation: DVPWA / Issues
- Filters: My Issues (All), Clear All Filters
- Filters Applied: Looking for Bugs, Vulnerabilities, or Code Smells? If your team prefers working with these types, change it in the [settings](#)
- Issues in new code:
 - Software Quality: 1 x
 - Security: 3
 - Reliability: 17
 - Maintainability: 65
 - Severity: 0
 - Blocker: 0
 - High: 0
- Issues List (Dockerfile.app):
 - Issue 1: Merge this RUN instruction with the consecutive ones. (Consistency, No tags, L10 • 5min effort • 7 years ago)
 - Issue 2: Replace this ADD instruction with a COPY instruction. (Consistency, No tags, L18 • 5min effort • 8 years ago)
 - Issue 3: Replace this ADD instruction with a COPY instruction. (Consistency, No tags, L19 • 5min effort • 8 years ago)
 - Issue 4: Replace this ADD instruction with a COPY instruction. (Consistency, No tags)

Wait if necessary

If the dashboard says **"Processing"**, wait 30–60 seconds and refresh. The server is still processing the uploaded analysis.